

WEB TECHNOLOGIES

CSE-4004



LECTURE 4: Functions



By: Dr.MD.Sirajuddin

Function

- A function is a reusable block of code that performs some specific task.
- A function is executed when it is called or invoked.

Function Declaration Syntax:

```
function function_name(param1, param2,...)  
{  
    statement(s);  
    return value; //optional  
}
```

Example: *function display()*
 {
 alert("Hello World");
 }

Function

Example:

display();

```
function display()  
{  
  alert("Hello World");  
}
```

**JavaScript hoists
function
declarations**

Function with return value

```
function display()
{
  return "Hello World";
}
let msg=display();
```

Function Expression: Function stored in a variable. Invoked using variable name. Also called **Anonymous Function**.

```
let result=function(a,b)
{
  return a+b;
}
alert(result(2,5));
```

Function Reference: Functions are objects

```
function display()
```

```
{
```

```
return " Hello World";
```

```
}
```

```
alert(display); // function reference - displays function object
```

```
alert(display()); // execute function - shows "Hello World"
```

Note:

- ✓ **Functions are objects**
- ✓ **Can be stored in variables**
- ✓ **Can be passed as arguments**
- ✓ **Can be returned from other functions**
- ✓ **Functions have properties (*length, name, prototype*)**

Accessing Function Properties

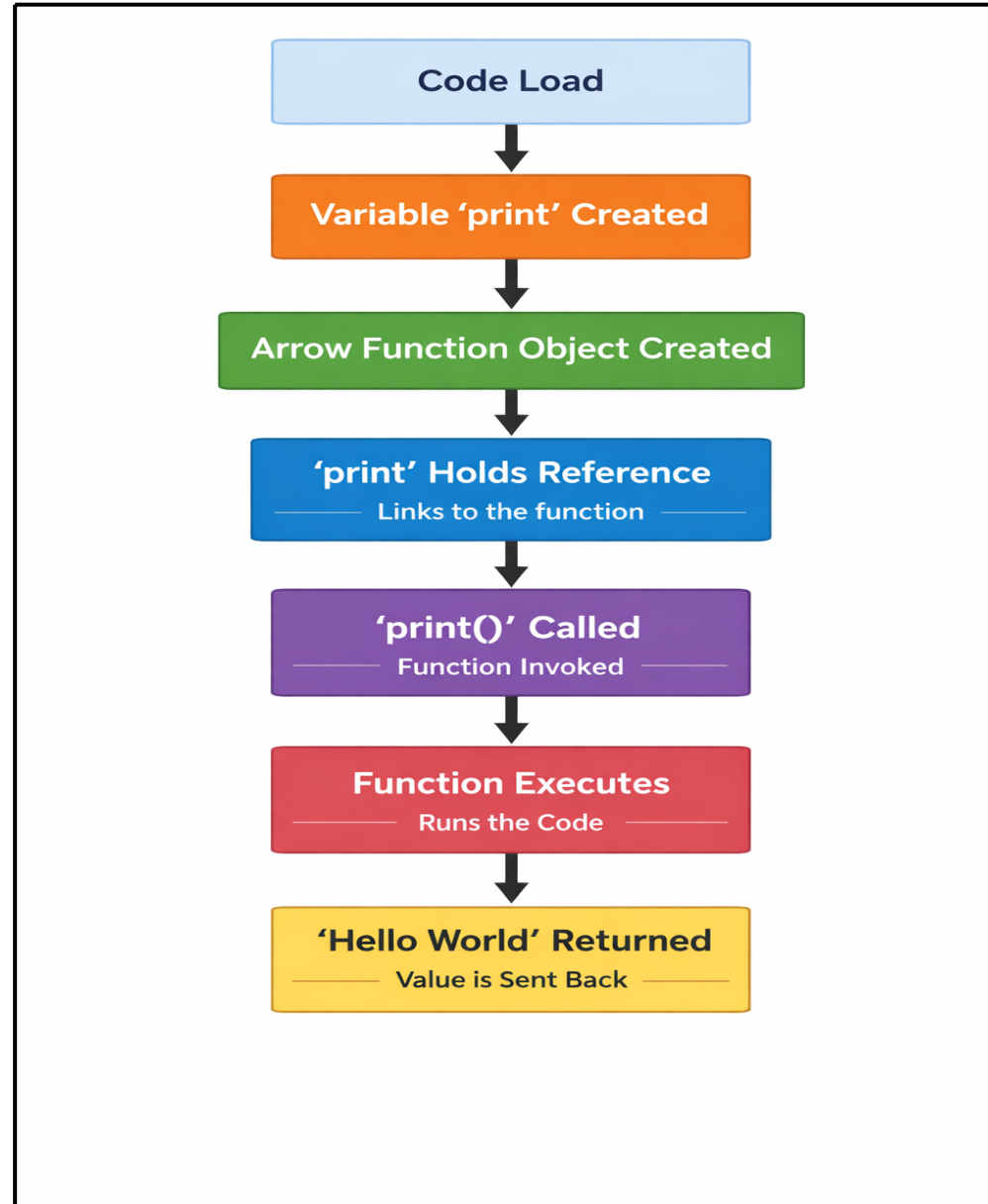
```
function display()  
{  
return "Hello World";  
}
```

```
alert(display.name); //function name  
alert(display.length); // no.of parameters  
alert(display.caller); // null if no caller
```

Arrow Function

- *Arrow function is a shorthand notation for writing function expression.*
- *It doesn't need **function** & **return** keywords and **Curley braces**.*
- *Arrow function is defined by using =>*
- *let add = (a,b) => a+b; // with parameters*
alert(add(1,2));
- *let cube = s => s*s*s; // without parenthesis*
alert(cube(2));
- *let print = () => "Hello World"; // without parameters*
alert(print());

Arrow Function Execution Process



IIFE: Immediately Invoked Function Expression

- Self invoked function
- Automatically executed when the JS engine reads it.

Syntax:

```
(function ()  
{  
    // Code to run immediately  
}) ();
```

Example:

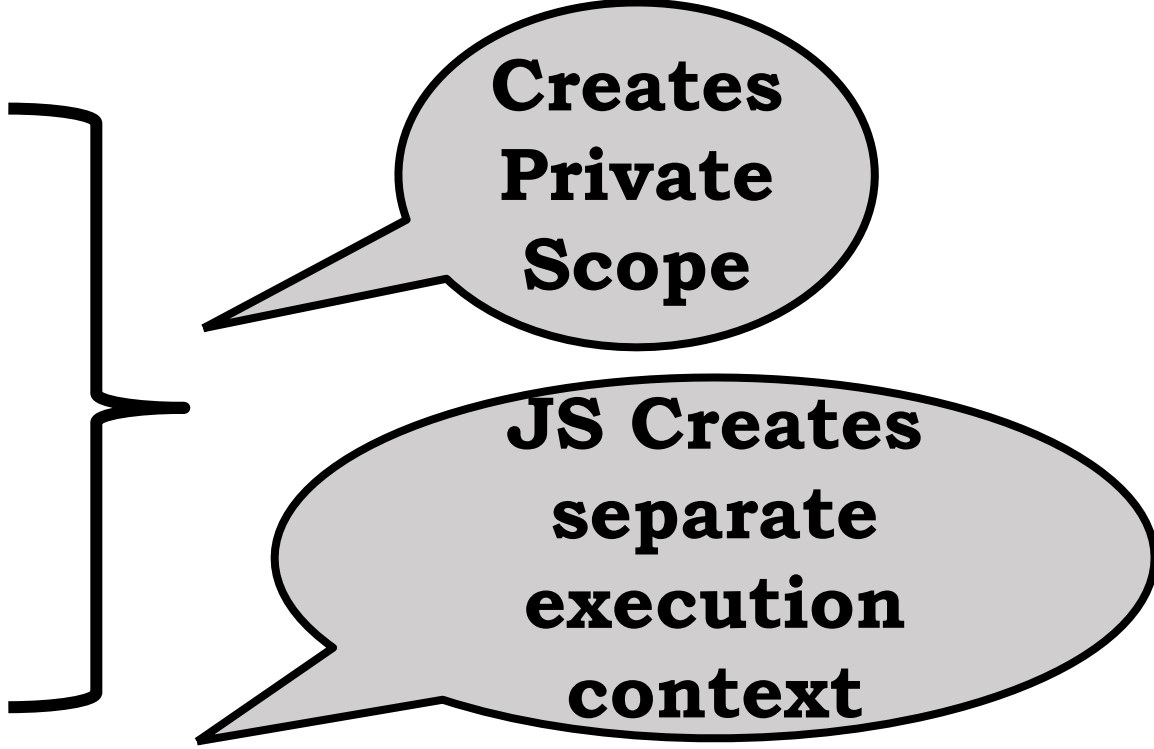
```
(function () {  
    alert("I am IIFE!!!, run immediately");  
})();
```

IIFE: Immediately Invoked Function Expression

Example:

```
(function ()  
{  
  let msg="Hello IIFE";  
  alert(msg);  
}) ();
```

alert(msg); **//error**



**Creates
Private
Scope**

**JS Creates
separate
execution
context**

Callback Function: A function passed as an argument

```
function user_input(display)  
{  
  const Institute="VIT-AP";  
  display(Institute);  
}
```

```
function showInstitute(name) // callback function  
{  
  alert("Institute Name:"+name);  
}
```

```
user_input(showInstitute);
```

RECURSIVE FUNCTIONS

- A recursive function is a function that calls itself until a specified condition is met.

Syntax:

```
function function_name()
{
    if (baseCondition)
    {
        return;          // stop recursion
    }
    function_name(); // function calls itself
}
```

Scenario

- **Imagine you are building an online shopping app. Every time user adds or deletes a product from cart, the cart total price should update.**

Scenario

```
let pro_Names = [ ];  
let pro_Prices = [ ];  
  
function add_Product(name, price) {  
    pro_Names.push(name);  
    pro_Prices.push(price);  
    alert(name + " added. Total: Rs" + compute_Total());  
}
```

Scenario

```
function remove_Product(name)  
{  
  const index = pro_Names.indexOf(name);  
  if (index !== -1)  
  {  
    pro_Names.splice(index, 1);  
    pro_Prices.splice(index, 1);  
    alert(name + " removed. Total: Rs." + compute_Total());  
  }  
  else  
  {  
    alert(name + " not found in cart.");  
  }  
}  
}
```

Scenario

```
function compute_Total()  
  {  
    let total = 0;  
    for (let price of pro_Prices)  
      {  
        total += price;  
      }  
    return total;  
  }
```